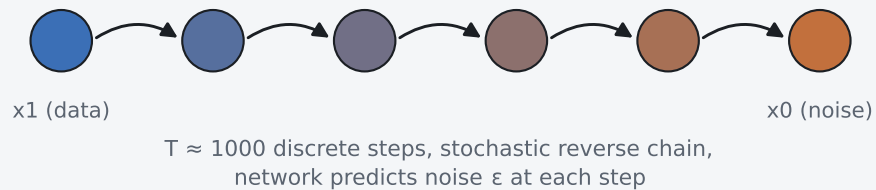


How the model works: flow matching, not DDPM

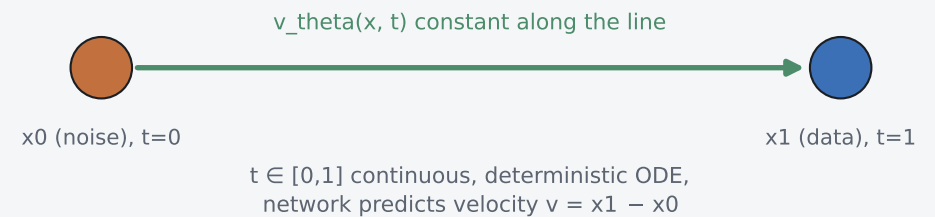
flow_model/ — a 3D U-Net trained to predict a velocity field, sampled by solving an ODE

This is conditional flow matching (the linear / "rectified flow" variant), not a DDPM. The package is literally named flow_model, the loss is flow_matching_loss (regress a predicted velocity onto $v^* = x_1 - x_0$ along straight lines from noise to data), and sampling/inversion solve an ODE rather than run a discrete denoising Markov chain. The only thing borrowed from DDPM is the shape of the time embedding (sinusoidal positional encoding) — a generic conditioning trick, not evidence of a diffusion process.

DDPM: discrete Markov chain



Flow matching: one straight line



DDPM

Forward process

Markov chain adding Gaussian noise, ~1000 steps

Training target

Predict noise ϵ (or score) added at step t

t

Integer step, 1..T

Sampling

Iteratively denoise via reverse SDE/chain

Stochasticity

Sampling is stochastic (unless DDIM-like)

This model (flow matching)

Single deterministic straight line, noise $\rightarrow$ data

Predict velocity $v = x_1 - x_0$ along the line

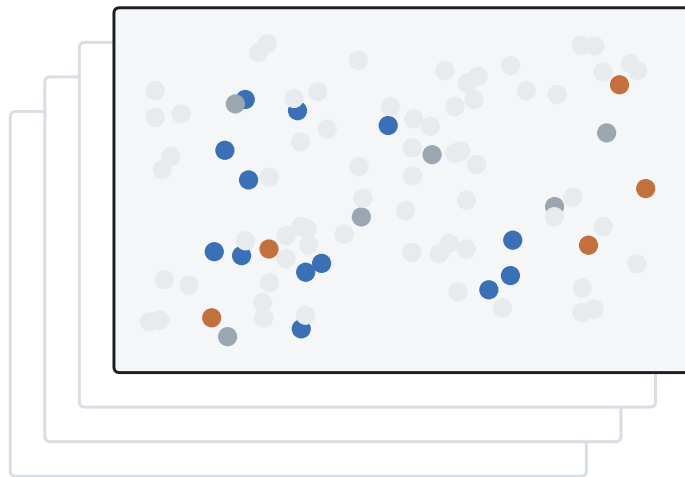
Continuous, $t \in [0, 1]$

Integrate $dx/dt = v_{\theta}(x,t)$, $t: 0 \rightarrow 1$

Sampling is deterministic (Euler/Heun ODE)

1. What the model generates

A one-hot 3D anatomical label volume (a synthetic MRI-like phantom), 4 classes per voxel:



x : [C=4, D=26, H=128, W=128]
one-hot over 4 classes per voxel

-  background
-  insert
-  pillar
-  lump

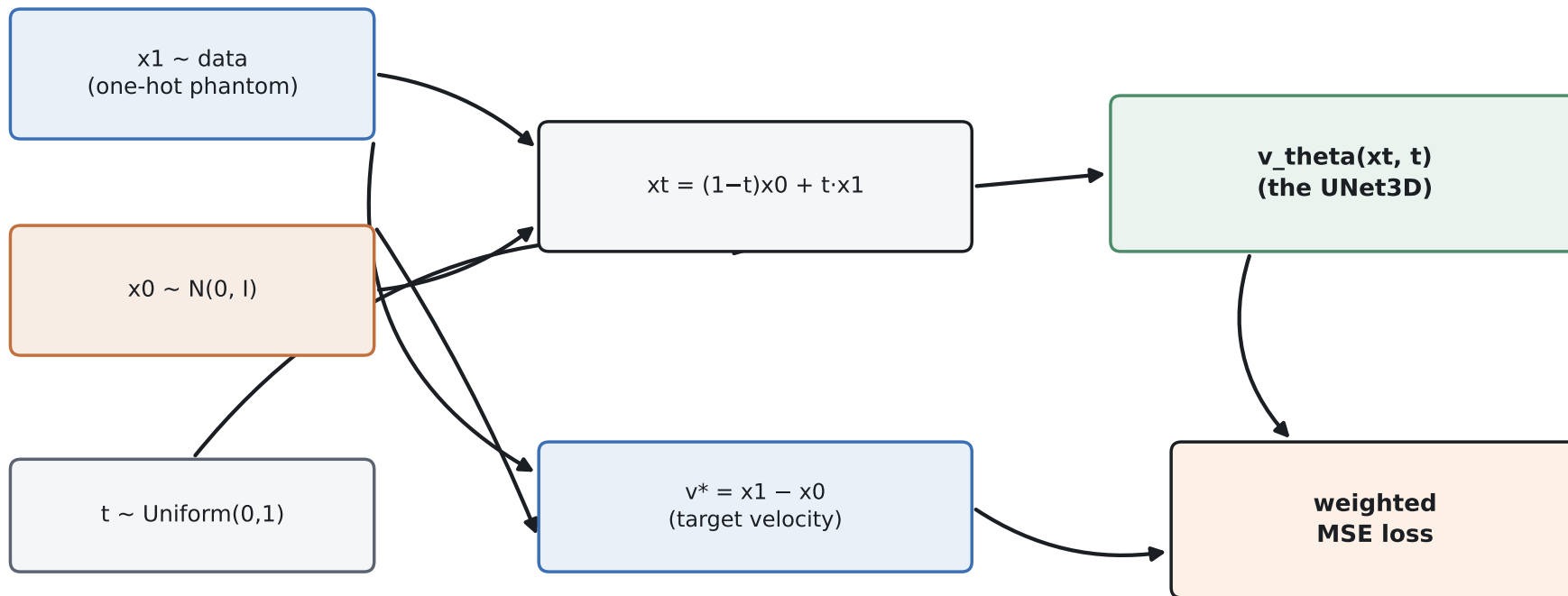
Training augments with 8 in-plane rotations (0° , 45° , ... 315°) per case, nearest-neighbor interpolation so labels stay exact integers (data.py: rotate_label_volume).

2. The conditional flow matching loss

$$x_t = (1-t) \cdot x_0 + t \cdot x_1$$

$$v^* = x_1 - x_0$$

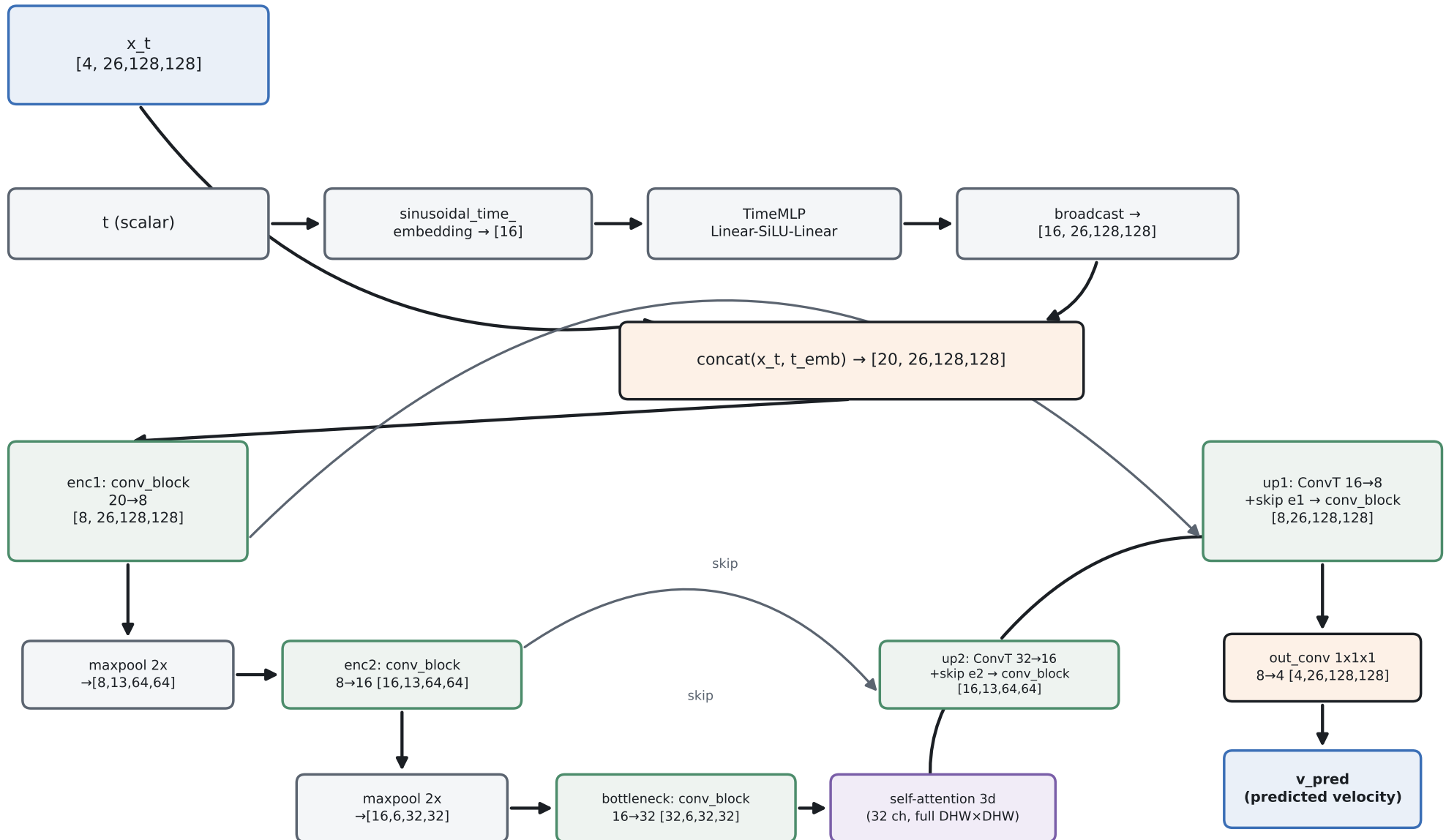
$$\text{loss} = \text{weighted_MSE}(v_{\text{theta}}(x_t, t), v^*)$$



Per-class weighted (background / insert / pillar / lump) because pillar/lump voxels are rare ($\sim 0.3\%$ / $\sim 0.9\%$ of all voxels) — unweighted MSE would let the model ignore them almost entirely (train.py: parse_class_weights).

3. FlowMatchingUNet3D architecture

default config: base_ch=8, embed_channels=16, num_classes=4



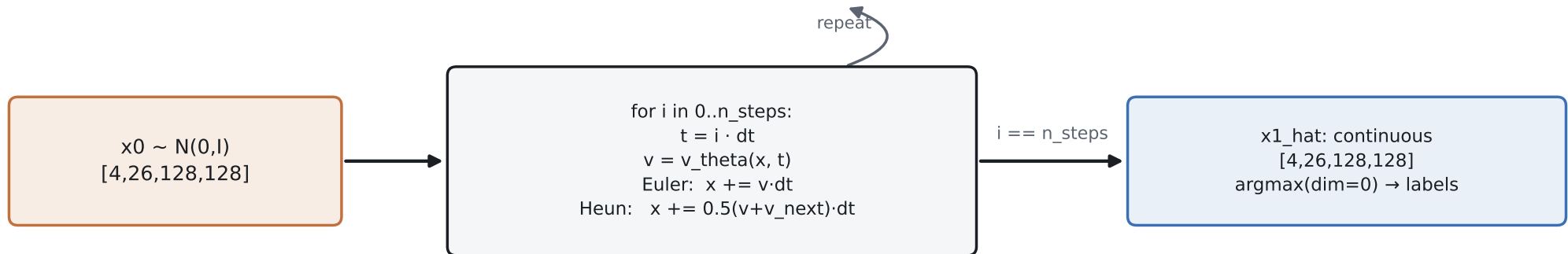
Notes: only 2 encoder / 2 decoder stages + bottleneck — deliberately small given dataset size and 3D memory cost.

Attention is full (non-windowed) self-attention over all $D \times H \times W$ bottleneck voxels — cheap since the bottleneck is only $6 \times 32 \times 32$.

up2 uses `output_padding=(1,0,0)` to undo the depth rounding from pooling $13 \rightarrow 6 \rightarrow 13$ instead of $12 \rightarrow 13$ (depth 26 is not a power of 2).

4. Sampling: noise → data (forward ODE)

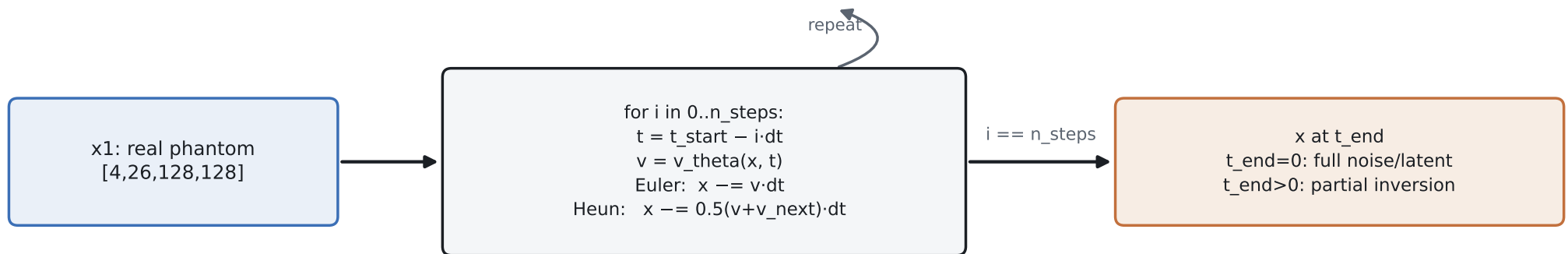
Integrates $dx/dt = v_theta(x, t)$ from $t=0$ (pure noise) to $t=1$ (data), Euler or Heun (RK2) steps (ode.py: sample)



Because v_theta was only ever trained on straight lines from $t=0$ to $t=1$, few steps ($n_train_sample_steps=30$ during training-time preview) already trace the path well — the classic flow-matching sampling-efficiency advantage over ancestral DDPM sampling.

5. Inversion: data $\rightarrow$ noise (backward ODE), and partial inversion

Runs the same learned ODE backward from a real phantom ($t=1$) toward noise ($t=0$); $t_{\text{end}} > 0$ stops partway — the latent still carries some of the original phantom's structure, which is what makes interpolation between two real phantoms possible (sweep_tmid.py, visualize_inversion.py) (ode.py: invert)



File map

File	Role
flow_model/config.py	All hyperparameters (Config dataclass + CLI overrides)
flow_model/data.py	Loads label volumes, one-hot encodes, rotation augmentation
flow_model/velocity_model.py	FlowMatchingUNet3D — time embedding + wraps UNet3D
flow_model/unet3d/	The 3D U-Net: conv_block_3d, self_attention_3d, up_conv_block_3d, unet_3d.py
flow_model/time_embedding.py	Sinusoidal t embedding (the one DDPM-style borrowing)
flow_model/train.py	Training loop + flow_matching_loss
flow_model/ode.py	sample() (forward ODE) and invert() (backward ODE)
flow_model/interpolate.py, sweep_tmids.py, visualize_inversion.py	Editing/interpolation experiments built on invert() + sample()

For the deeper theoretical discussion (why flow matching has no compression bottleneck, how that shapes the interpolation-quality experiments, and the latent-diffusion proposal to fix it), see [latex/project_summary.pdf](#) / [latex/project_summary.tex](#) — this document is the mechanics-level companion to that.